

INTRODUCTION TO AWS CLOUD

What is Cloud Computing?

- On-demand consumption of computing/IT resources and services via the Internet using a pay-as-you-go model
 - **On demand:** Fast provisioning of IT resources and services when needed
 - **Accessible from the Internet:** Can be provisioned and used from anywhere in the world
 - **Pay-as-you-go model:** You only pay for what you use (e.g. based on hourly use or amount of storage space used)

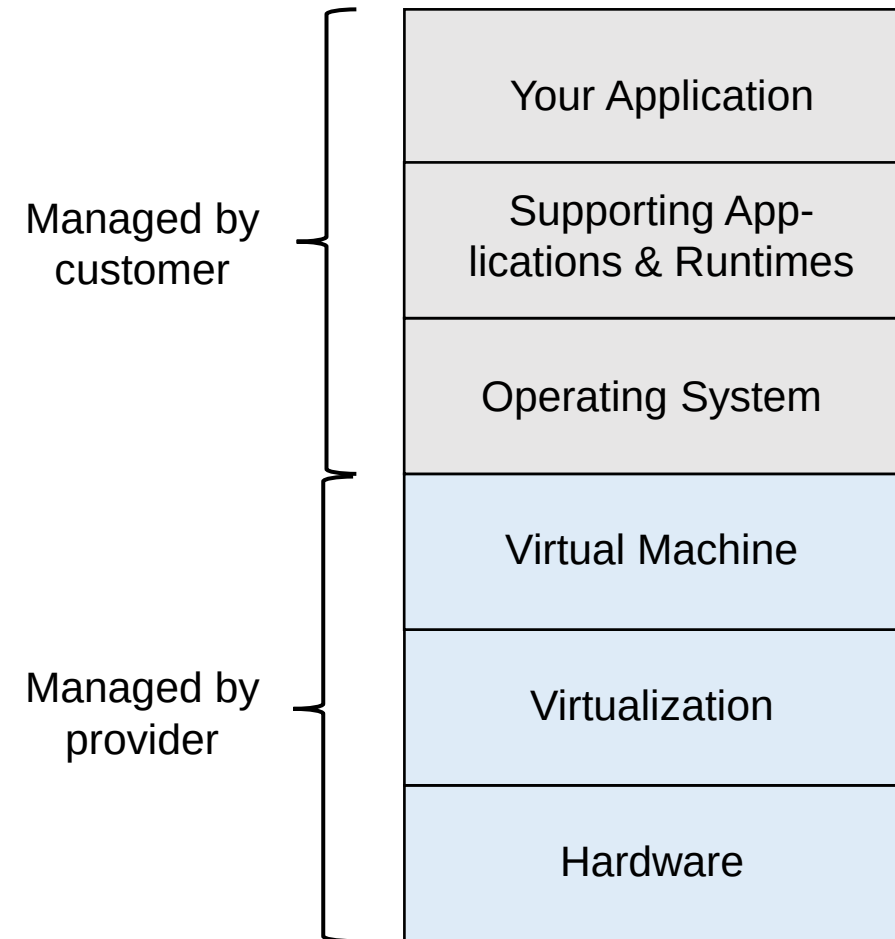
Advantages of Cloud Computing

- **Flexible and agile business development:** No need to order and rack physical servers and networking devices or build a server room/data center (building, power, cooling, physical security, IT staff), which may take months or years
 - Easy to scale up (out) or down (in): No need to predict future capacity or over-engineer the solution to match the peak demand (e.g. a web store for Christmas time)
- **Moving from CAPEX to OPEX:** No need to make large up front capital investment
 - Easy to experiment: low risk associated with starting new projects/products/companies

Cloud Computing Service Models (1/3)

- **Infrastructure as a Service (IaaS)**

- Provides compute, networking and storage resources
- The user manages the OS, all supporting applications and runtimes (e.g. data base or web container, and JRE/JEE runtimes), and the application (e.g. Java web store application)
- E.g. compute (VM) service, virtual private cloud service



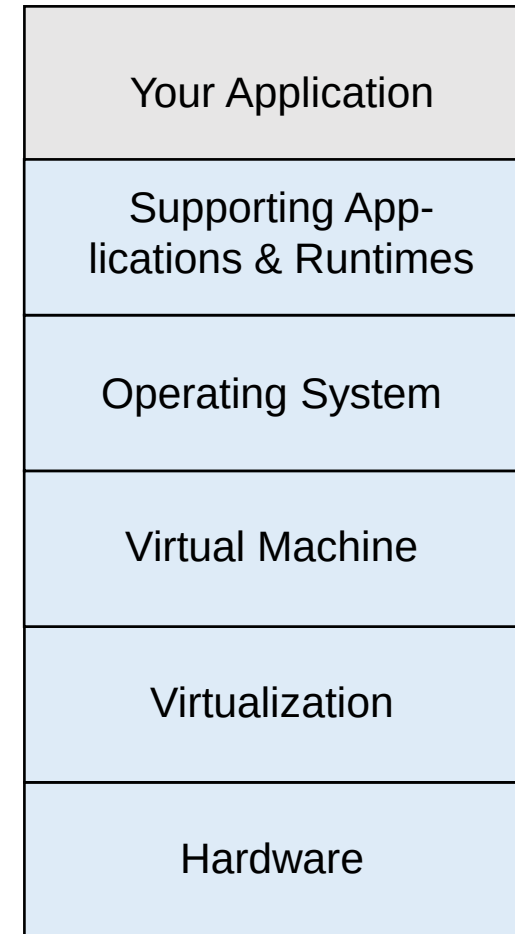
Cloud Computing Service Models (2/3)

- **Platform as a Service (PaaS)**

- Provides compute, networking and storage resources, and manages the OS, all supporting applications and runtimes (e.g. database)
- The user manages the application (e.g. Java web store application)
- E.g. Relational database service, app deployment service

Managed by
customer

Managed by
provider

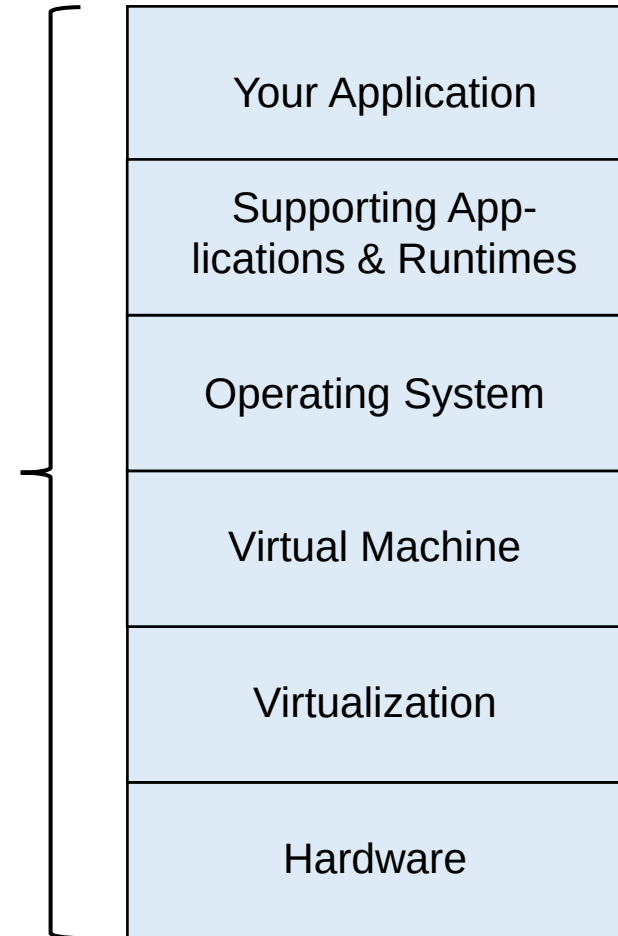


Cloud Computing Service Models (3/3)

- **Software as a Service (SaaS)**

- Provides compute, networking and storage resources, manages the OS, all supporting applications and runtimes (e.g. database), and the application (e.g. Java web store application)
- E.g. Google apps, Microsoft Office 365

Managed by
provider



Cloud Computing Deployment Models

- **Public cloud:**

- Cloud service managed by an organization and open to use by the general public (e.g. AWS, MS Azure, Google Cloud, ...)

- **Private cloud:**

- Cloud service managed by an organization and available only within that organization (e.g. on-premise data center with virtualization and automated service provisioning)

- **Hybrid cloud:**

- Mixture of public and private cloud (some applications hosted in a local data center and some applications hosted by a public cloud service)

AWS Global Infrastructure (1/3)

- **Regions:**

- Geographic location where AWS services are available
- Consists of two or more availability zones (typically 3)

- **Availability Zones (AZs):**

- Consist of 1-6 data centers
- Each AZ has redundant power supply and networking connectivity
- AZs are designed to be independent of other AZs (e.g. located in different floodplains, connected to different power grids, ...) so that outages/natural disasters in one AZ do not affect other AZs in the region

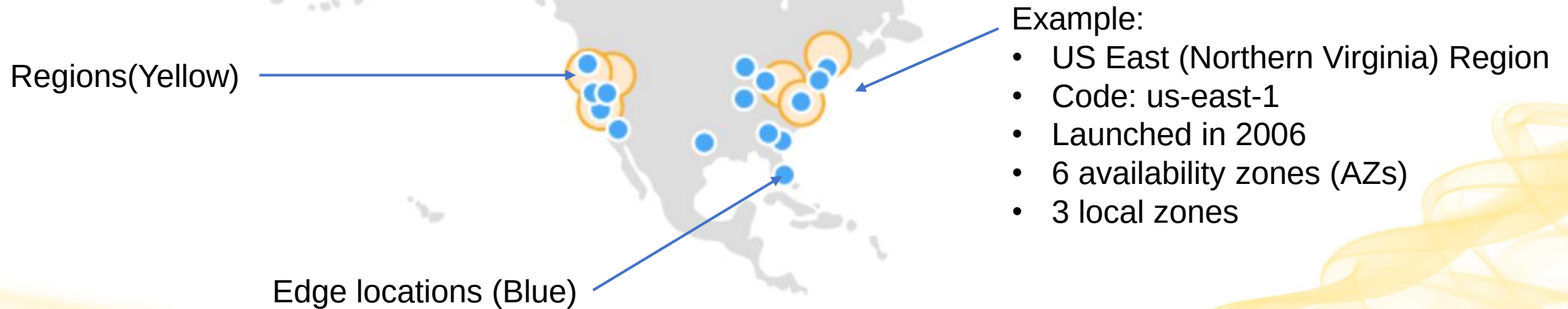
AWS Global Infrastructure (2/3)

- **Local Zones:**
 - Smaller data centers that run a few selected services (compute, storage, ...) close to large population or industry centers where no regions exist
 - Connected to the parent region via high-bandwidth private network
- **Edge Locations (also called Points of Presence/POPs):**
 - Amazon's content delivery network (CDN)
 - Web caches delivering static content (e.g. video or CSS files) to local users to reduce network latency

AWS Global Infrastructure (3/3)

- Region map, North America

6 Regions



AWS Products and Services (1/11)

- **Compute:** some examples (not exhaustive)
 - **Elastic Compute Cloud (EC2):**
 - Provisions virtual machines or **instances**
 - Large variety: general purpose, compute-optimized, memory-optimized, storage-optimized, GPU-optimized, ...
 - **Auto Scaling:**
 - Dynamically scales the number of EC2 instances up and down depending on the current load
 - Controlled by user-defined policies
 - E.g. starts new instances if the average CPU utilization of the currently running instances exceeds 80%

AWS Products and Services (2/11)

- **Compute (cont.):**
 - **Lambda: “serverless” applications**
 - Deploys and runs code without having to provision and manage EC2 instances
 - Can be used to respond to event triggers (e.g. file upload to S3)
 - Provides automatic scalability and high-availability
 - **EC2 Container Service (ECS):**
 - Runs Docker containers on EC2 instances
 - No need to install and manage container management software

Elastic Compute Cloud (EC2) (1/2)

- Provisions **compute capability** (virtual machines) in the cloud
- **Main steps to launch an EC2 instance:**
 1. Choose an Amazon Machine Image (AMI): OS and pre-installed SW
 2. Choose an instance type: HW configuration (e.g. number of vCPUs, amount of memory etc.)
 3. Configure the instance details (e.g. network settings, shutdown behavior etc.)
 4. Select storage (e.g. storage type, volume size, etc.)

Elastic Compute Cloud (EC2) (2/2)

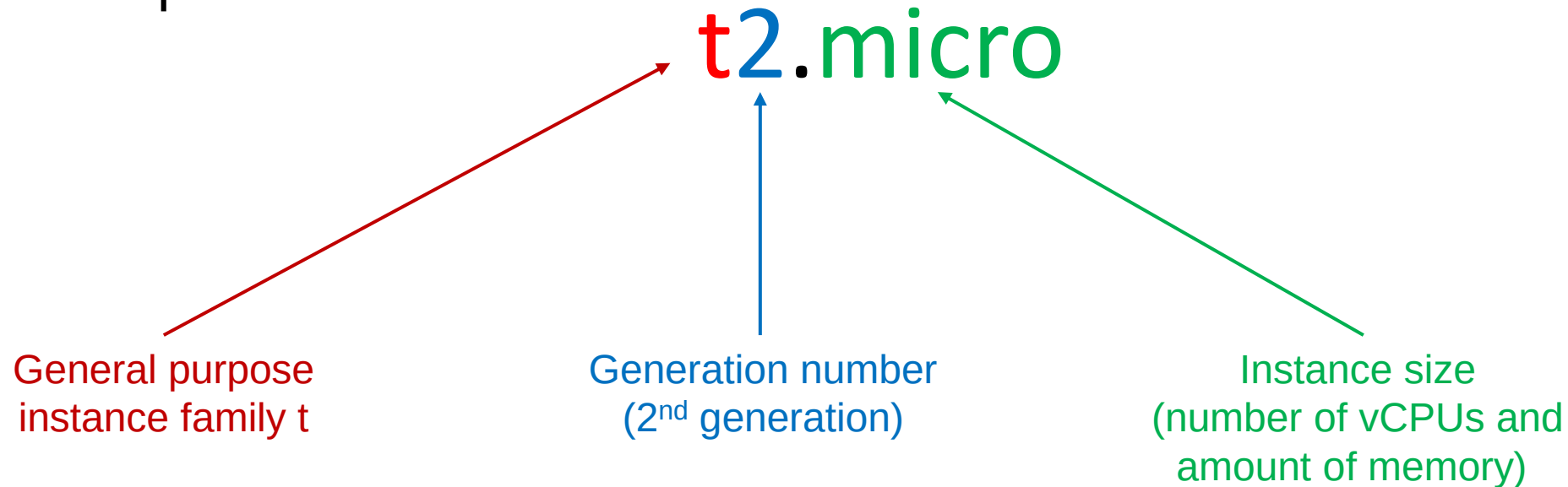
- **Main steps to launch an EC2 instance (cont.):**
 5. Add tags: key/value pairs to label the instances (e.g. key: “Name”, value: “myTestInstance”)
 6. Configure security groups: firewall rules for the instance (e.g. pass through SSH traffic, block everything else)
 7. Select/create a key pair for SSH access (password-based access is disabled)
 8. Launch instance

Amazon Machine Image (AMI)

- Virtual machine template for the AWS cloud
- Bundle of OS and pre-installed SW packages (e.g. Python interpreter) – may incur additional charges
- AMI sources:
 - **AMIs offered by AWS**
 - Supported OSs: Windows and Linux (Ubuntu, CentOS, Red Hat Enterprise Linux, SUSE, Amazon Linux)
 - **AWS Marketplace AMIs**: third party providers (e.g. Barracuda firewall, Juniper Networks vSRX, ...)
 - **Community AMIs**: provided by the community
 - *Not verified by AWS*: may contain bugs or other problems

EC2 Instance Types (1/5)

- Determines the HW features of the EC2 instance
- Example instance:



EC2 Instance Types (2/5)

- **Instance families:**

- **General purpose:** balance of compute, memory and networking resources
 - Instance families: t3, t3a, t2, m6g, m5, m5a, m5n, m4, a1
 - t3 and t2 provide burstable performance based on accrued CPU credits
 - Use cases: web servers, development and test environments

EC2 Instance Types (3/5)

- **Instance families (cont.):**
 - **Compute optimized:** for heavy computational workloads
 - Instance families: c6g, c5, c5a, c5n, c4
 - Use cases: media transcoding, high-performance computing, long-running batch jobs, gaming servers
 - **Memory optimized:** for large memory requirements
 - Instance families: r6g, r5, r5a, r5n, r4, x1e, x1, z1d
 - Use cases: processing large data sets in memory, in-memory databases, NoSQL databases, Apache Spark, genome analysis

EC2 Instance Types (4/5)

- **Instance families (cont.):**
 - **Storage optimized:** high-speed access to large data sets stored on disk/SSD
 - Instance families: i3, i3n, d2, h1
 - Use cases: relational databases, NoSQL databases, data warehouse applications, Hadoop/MapReduce
 - **Accelerated computing:** access to HW accelerators such as GPUs and FPGAs
 - Instance families: p3, p2, inf1, g4, g3, f1
 - Use cases: machine learning algorithms, molecular modeling, graphics processing, computational finance

EC2 Instance Types (5/5)

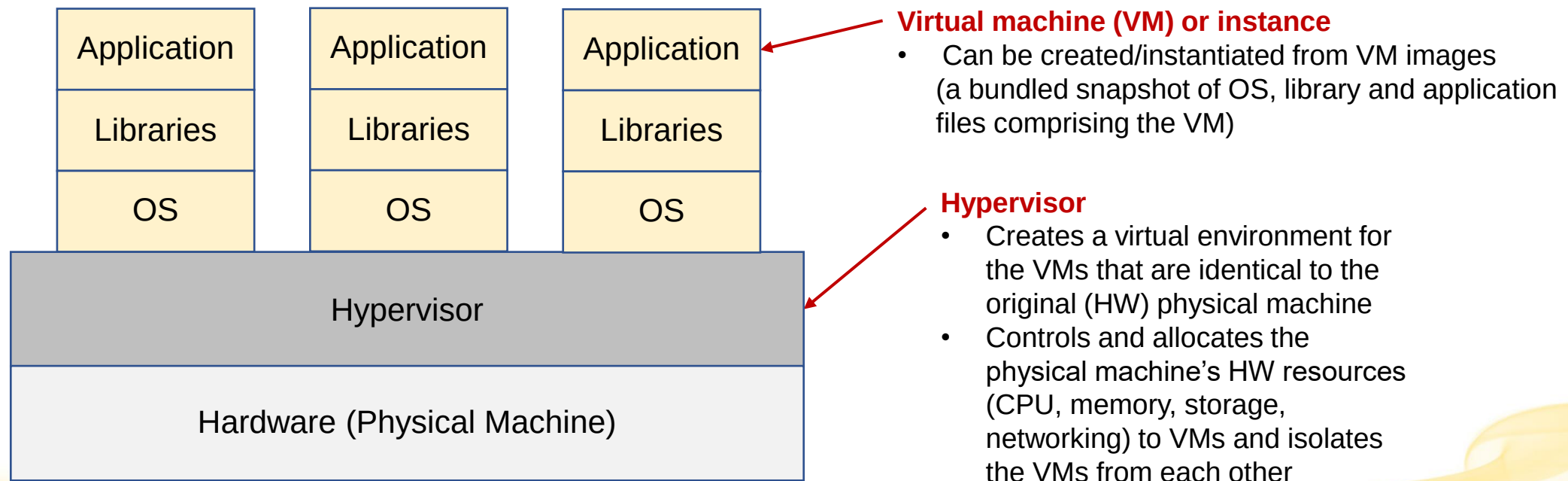
- **Instance sizes:**
 - From **nano** to **16xlarge** with more and more vCPUs and memory
 - For example: **t2 family**

Free tier →

Instance	Number of vCPUs	Memory (GB)
t2.nano	1	0.5
t2.micro	1	1
t2.small	1	2
t2.medium	2	4
t2.large	2	8
t2.xlarge	4	16
t2.2xlarge	8	32

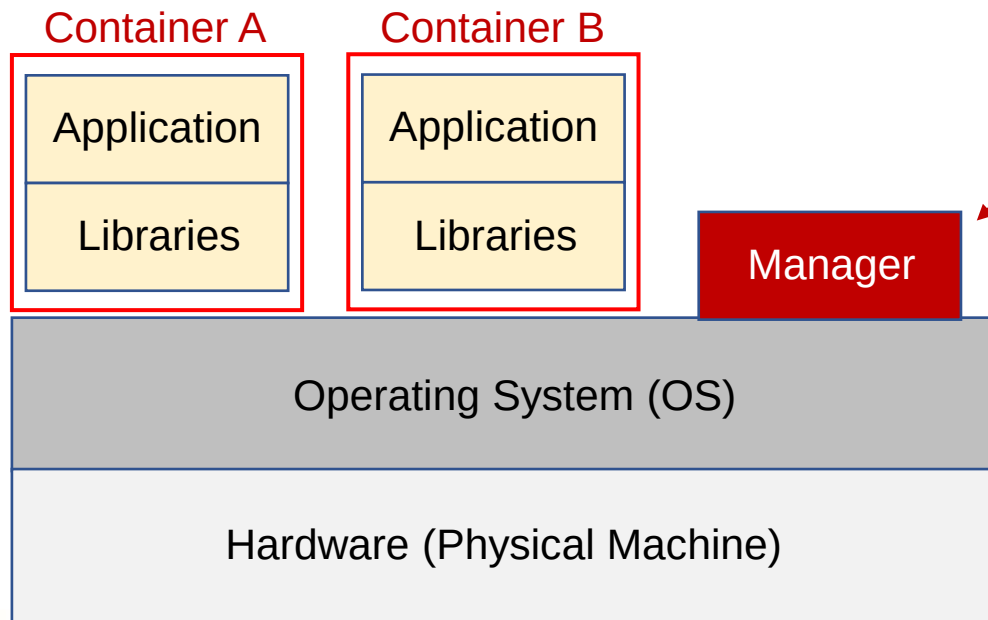
Virtual Machines

- Dividing a physical machine into multiple virtual machines via a SW application where each virtual machine runs its own OS



Container Technology

- Managing and running SW packages containing one or more application(s) and their dependencies



Container manager (e.g. Docker server)

- Uses Linux name spaces and control groups to allocate resources (such as CPU, memory, networking) and to isolate processes belonging to different containers
- Creates and runs containers from a container image (a bundled snapshot of all files needed to run the applications in the container)

Difference Between VMs and Containers

Virtual Machines	Containers
Complete OS/library/application stack running on virtual hardware	Wrapper and management layer around some processes
Each VM running on the same physical machine may have a different OS (Linux, Windows, ...)	All containers running on the same physical machine share the same OS kernel
Heavy footprint (VM image: several GB)	Light footprint (container image: few MB)
Slow to start (several minutes)	Quick to start (few seconds)
Long-lived instances (several days/weeks/months)	Short-lived instances (can run only for a few minutes)
Provides better isolation between different tenants/users (more secure)	Provides limited isolation between different tenants/users (less secure)